Dalhousie University
CSCI 3110 — Design and Analysis of Algorithms I
Fall 2024
Assignment 5

Distributed Tuesday, October 15 2024.
Due 11:59PM, Tuesday, October 29 2024.

Instructions:

1. Before starting to work on the assignment, make sure that you have read and understood policies regarding the assignments of this course, especially the policy regarding collaboration. `https://dal.brightspace.com/d2l/le/content/338900/Home?itemIdentifier=D2L.LE.Content.ContentObject.ModuleCO-4457275`

2. To submit via Crowdmark, upload a **separate** image/PDF file (multiple pages are allowed) to each question. You may resubmit as often as necessary until the due date. A good strategy is to create an initial submission days in advance after you solve some problems, and make resubmissions later. More detailed submission information for Crowdmark can be found at:

   `https://crowdmark.com/help/completing-and-submitting-an-assignment/`

3. If you submit a joint assignment, use Crowdmark to create a group and add group members before making submissions. The "What Will Students See" section of `https://crowdmark.com/help/creating-a-group-assignment` shows the steps and screenshots of creating a group and adding members. Note that, "to avoid overwriting submissions", Crowdmark recommends "only one group member submits all the work".

4. We encourage you to typeset your solutions using LaTeX. However, you are free to use other software or submit scanned handwritten assignments as long as **they are legible**. We have the right to take marks off for illegible answers.

Questions:

1. [10 marks] In class, we learned that the running time of Karatsuba's algorithm can be expressed as the recurrence $T(n) = 3T(n/2) + n$. We then used the substitution method to solve the recurrence.

   When using the substitution method, it seems difficult to make a guess about the upper bound on the running time. However, if we use the recursion tree method, it would be a lot easier.

In this question, you are asked to solve this recurrence using the recursion tree approach, which can be further used to help us make the guess (you are not required to explain how to make use of your result to make the guess; only show your work using the recursion tree). You can assume that $T(1) = 1$ and that n is a power of 2 for convenience. Give your solution using big-Oh notation.

2. [10 marks] *The Story:* In ancient times, there were hundreds of city states in Greece. Sometimes these city states banded together to fight other countries, but in other times, they often fought each other over borders. Among them, Sparta and Messenia shared a long border, and they fought multiple wars against each other.

   Suppose, after one of these wars, the kings of Sparta (Sparta was an oligarchy, so I used the plural form of "king") decided to build a set of watchtowers to guard the borderline. Each watchtower would be responsible for watching over a section of the borderline.

   The Spartan military thus surveyed the borderline and identified a set of possible locations. Each location can be used to construct one watchtower. For each possible location, they also determined the (continuous) section of the borderline that could be guarded by the watchtower built on this location. They verified that, if they built a watchtower at each location, it would be sufficient to cover the entire borderline. However, since the sections guarded by some of these watchtowers would overlap with each other, it was possible to select a subset of these locations to construct enough watchtowers to guard the entire borderline. To save costs, it would be ideal to build as few watchtowers as possible, while still guarding the entire borderline.

   The Model: Mathematically, this borderline could be viewed as a simple curve; a simple curve is a curve that does not cross itself. Let A and B be the two endpoints of this curve, and let m be its length in Greek feet (Greek foot is called pous, and 1 Greek foot is equivalent to 0.308 meters).

   Furthermore, we identify each point in this curve by the length of the subcurve between this point and the endpoint A. Thus, the section of the borderline that could be guarded by a watchtower is represented by a closed interval $[s, f]$; this means that this watcher tower could guard the section for the borderline between and including points S and F, where the length of the section of the borderline between S and A is s Greek feet, the length of the section of the borderline between F and A is f Greek feet, and $s < f$. Then the length of the section of the borderline guarded by this watchtower is $f - s$, which is also the length of the interval.

   Thus, the input to this problem is a set, I, of n intervals, and each interval in I represents the section of the borderline that can be guarded by the watchtower at a certain location. The union of all intervals in I is $[0..m]$. The output is a subset, I', of I, of minimum cardinality such that the entire borderline is guarded by the watchtowers corresponding to the selected intervals in I'.

Your first task: Suppose that, the Spartan kings hired you, a time traveler who happened to be in Ancient Greece at that time, to solve this problem. They also forwarded you one solution proposed by the Spartan Military: First select the location that could guard the section of the borderline of the longest length, and build a watchtower at this location. Then, among all remaining locations where watchtowers could still guard some of the unguarded portions of the borderline, select the location whose corresponding interval in I is the largest. Repeat this process.

For example, if $m = 13$, and $I = \{[3, 10], [7, 13], [6, 11], [0, 4]\}$, then their strategy would first choose $[3, 10]$, then $[7, 13]$ and finally $[0, 4]$.

Your first task is to show that this strategy does not always give an optimal solution by constructing a counterexample. Explain why the solution is not optimal.

3. [10 marks] Design an algorithm for the problem described in Question 2 that *always* finds an optimal solution in $O(n \lg n)$ time.

   When writing down the pseudocode, you could assume that I is an array in which each entry stores the two endpoints of an interval.

   When you justify the correctness of your algorithm, carefully prove that your greedy strategy always gives an optimal solution.

   Hint: You may be able to think of a correct greedy strategy whose obvious implementation uses $O(n^2)$ time. This solution, if properly proven, described and analyzed, should be worth most of the marks, and to obtain full marks, focus on how to implement it efficiently.